

```

unlock_kernel();
return put_user(res, p);
}

case FIGETBSZ:
    if ((inode->i_sb == NULL))
        return -EBADF;

    return put_user(inode->i_sb->s_blocksize, p);
case FIONREAD:
    return put_user(f_size_read(inode) - filp->i_pos, p);
}

return do_ioctl(filp, cmd, arg);
}

```

Networking

Today, we will look at some of the fundamental functionalities (that can handle network services) required when you code. Since some of our readers may not have a strong background in networking, we will spend some time reviewing the basic networking related concepts.

In Linux, you can use sockets for accessing network services. And you can employ the following functionalities to do higher-end tasks:

```

int socket(int addr_family, int type, int protocol);
int bind(int s, struct sockaddr *address, int address_len);
int listen(int s, int backlog);
int connect(int s, struct sockaddr *address, int address_len);
int accept(int s, struct sockaddr *address, int *address_len);
int send(int s, char *msg, int len, int flags);
int sendto(int s, char *msg, int len, int flags, struct sockaddr *to, int tolen);
int recv(int s, char *buf, int len, int flags);
int recvfrom(int s, char *buf, int len, int flags, struct sockaddr *from, int fromlen);
int getsockopt(int s, int level, int option, char *ovalue, int *olen);
int setsockopt(int s, int level, int option, char *ovalue, int *olen);

```

The above code shows the set of C library routines that are included in the interface. It is important to note that these functions rely on the system call *socketcall*. The socket function is initiated by the following code:

```

static int sock_no_open(struct inode *irrelevant, struct file *dontcare);
static ssize_t sock_aio_read(struct kiocb *iocb, char __user *buf,
    size_t size, loff_t pos);
static ssize_t sock_aio_write(struct kiocb *iocb, const char __user *buf,
    size_t size, loff_t pos);
static int sock_mmap(struct file *file, struct vm_area_struct * vma);

static int sock_close(struct inode *inode, struct file *file);
static unsigned int sock_poll(struct file *file,
    struct poll_table_struct *wait);
static long sock_ioctl(struct file *file,

```

```

    unsigned int cmd, unsigned long arg);
static int sock_fasync(int fd, struct file *filp, int on);
static ssize_t sock_read(struct file *file, const struct iovec *vector,
    unsigned long count, loff_t *ppos);
static ssize_t sock_write(struct file *file, const struct iovec *vector,
    unsigned long count, loff_t *ppos);
static ssize_t sock_sendpage(struct file *file, struct page *page,
    int offset, size_t size, loff_t *ppos, int more);

static struct file_operations socket_file_ops = {
    .owner = THIS_MODULE,
    .llseek = no_llseek,
    .aio_read = sock_aio_read,
    .aio_write = sock_aio_write,
    .poll = sock_poll,
    .unlocked_ioctl = sock_ioctl,
    .mmap = sock_mmap,
    .open = sock_no_open, /* special open code to disallow open via /proc */
    .release = sock_close,
    .fasync = sock_fasync,
    .readv = sock_readv,
    .writev = sock_writev,
    .sendpage = sock_sendpage
};

```

Various functions like *net_family_write_lock* are also included:

```

static void net_family_write_lock(void)
{
    spin_lock(&net_family_lock);
    while (atomic_read(&net_family_locked) != 0) {
        spin_unlock(&net_family_lock);
        yield();
        spin_lock(&net_family_lock);
    }
}

```

As I said, since networking in Linux is a vast subject, we will be dedicating the next article entirely to it. I will also be briefing readers about the basic concepts required to meddle with networking in Linux. I would recommend that you refer to an undergraduate module in networking, if you don't have a clear picture. I will be including concepts related to the layer architecture models, protocols, conversion algorithms, etc, in the next column.

Happy kernel hacking! 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com